1.

```
public class Main {
    public static void main(String[] args) {
        String s1 = "Java";
        String s2 = "Java";
        String s3 = new String("Java");

        System.out.println(s1 == s2);
        System.out.println(s1 == s3);
        System.out.println(s1.equals(s3));
    }
}
```

A)

true
false
true

B)

true
true
true

C)

false
false
true

D) Compilation Error

2.

```
public class Main {
    public static void main(String[] args) {
        try {
            System.out.print("A");
```

```
            return;
        }
        finally {
            System.out.print("B");
        }
    }
}
```

A) A

B) AB

C) B

D) Runtime Error

3.

```
public class Main {
    public static void main(String[] args) {
        int x = 5;
        System.out.println(x++ + ++x);
    }
}
```

A) 11

B) 12

C) 13

D) 14

4.

```
Integer a = 100;
Integer b = 100;

Integer c = 200;
Integer d = 200;
```

```
System.out.println(a == b);
System.out.println(c == d);
```

A)

```
true
true
```

B)

```
true
false
```

C)

```
false
true
```

D)

```
false
false
```

5.

```
int[] a = {1,2,3};
int[] b = a;

b[1] = 10;

System.out.println(a[1]);
```

A) 2

B) 10

C) 3

D) Error

6.

```
class Test{
    void show(int x){
        System.out.print("int ");
    }

    void show(double x){
        System.out.print("double ");
    }

    public static void main(String args[]){
        Test t=new Test();
        t.show(5);
        t.show(5.5f);
    }
}
```

A)

int int

B)

int double

C)

double double

D) Error

7.

```
String s = "Hello";
s.concat(" World");
```

System.out.println(s);

A) Hello World

B) Hello

C) World

D) Error

8.

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a == b);
System.out.println(a.equals(b));
```

A)

true
true

B)

false
true

C)

false
false

D)

true
false

9.

```
class A{
   A(){
      System.out.print("A");
   }
}

class B extends A{
   B(){
      System.out.print("B");
   }

   public static void main(String args[]){
      new B();
   }
}
```

A) AB

B) BA

C) A

D) B

10.

```
class Test{
   static{
      System.out.print("A");
   }

   public static void main(String args[]){
      System.out.print("B");
   }
}
```

A) BA

B) AB

C) A

D) B

11.

```
int arr[] = new int[5];
System.out.println(arr.length);
```

A) 4

B) 5

C) 0

D) Error

12.

```
char c='A';
System.out.println(c+1);
```

A) A1

B) B

C) 66

D) Error

13. continue

```
for(int i=1;i<=5;i++){

   if(i==3)
      continue;

   System.out.print(i);
}
```

A) 12345

B) 1245

C) 345

D) 12435

14.

```
for(int i=1;i<=5;i++){

   if(i==3)
      break;

   System.out.print(i);
}
```

A) 12345

B) 12

C) 45

D) 123

15.

```
try{
   int x=10/0;
}
catch(ArithmeticException e){
   System.out.println("Handled");
}
```

A) Runtime Error

B) Handled

C) Exception

D) Nothing

16.

```
class A{
   void show(){
      System.out.print("A");
   }
}

class B extends A{
   void show(){
      System.out.print("B");
   }

   public static void main(String args[]){
      A obj=new B();
      obj.show();
   }
}
```

A) A

B) B

C) AB

D) Error

17.

```
StringBuilder sb=new StringBuilder("Hi");

sb.append(" Java");

System.out.println(sb);
```

A) Hi

B) Java

C) Hi Java

D) Error

18.

String s=null;

System.out.println(s.length());

A) 0

B) NullPointerException

C) Error

D) null

19.

System.out.println(10/4);

A) 2

B) 2.5

C) 3

D) Error

20. Floating Division

System.out.println(10/4.0);

A) 2

B) 2.5

C) 3

D) 2.0

21.

```
ArrayList<String> list=new ArrayList<>();

list.add("A");
list.add("B");
list.remove("A");

System.out.println(list);
```

A)

[A]

B)

[B]

C)

[]

D) Error

22.

```
HashSet<Integer> set=new HashSet<>();

set.add(10);
set.add(20);
set.add(10);
```

System.out.println(set.size());

A) 2

B) 3

C) 1

D) Error

23.

```
int arr[]={2,4,6};

for(int x:arr){
    x=x+1;
}

System.out.println(arr[1]);
```

A) 4

B) 5

C) 6

D) Error

24.

```
int x=2;

switch(x){

case 1:System.out.print("A");
case 2:System.out.print("B");
case 3:System.out.print("C");

}
```

A) B

B) BC

C) ABC

D) C

25.

```
final int x=10;

x=20;

System.out.println(x);
```

A) 20

B) 10

C) Runtime Error

D) Compilation Error

26.

```
class Test{

static int x=0;

Test(){
x++;
}

public static void main(String args[]){

new Test();
new Test();

System.out.println(x);
```

```
}
}
```

A) 0

B) 1

C) 2

D) Error

27.

```
class A{

static void show(){
System.out.print("A");
}
}

class B extends A{

static void show(){
System.out.print("B");
}

public static void main(String args[]){
A obj=new B();
obj.show();
}
```

A) A

B) B

C) AB

D) Error

28.

```
Integer x=10;

int y=x;

System.out.println(y);
```

A) 10

B) Error

C) Runtime Exception

D) null

29.

```
interface A{
int add(int x,int y);
}

public class Main{

public static void main(String args[]){

A obj=(x,y)->x+y;

System.out.println(obj.add(5,6));

}

}
```

A) 10

B) 11

C) Error

D) Runtime Exception

30.

What is wrong with this code?

```
public class Main{

public static void main(String args[]){

String s="Java";

if(s=="Java")
System.out.println("Equal");

}
}
```

A) Compilation Error

B) Runtime Error

C) Nothing is wrong; prints `Equal`

D) Must use `.equals()`; == never works for strings

ANSWERS:

1. String Pool

```
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");
```

Answer: A

true
false
true

Explanation

- `s1` and `s2` point to the same String Pool object.
- `new String()` creates a new object.
- `.equals()` compares content.

2. finally Block

```
try{
  return;
}
finally{
  System.out.print("B");
}
```

Answer: B

AB

Explanation

- `finally` always executes even if `return` is encountered.

3. Increment Operators

```
int x=5;
System.out.println(x++ + ++x);
```

Answer: B

12

Explanation

Initial

x = 5

x++

- uses 5
- x becomes 6

++x

- x becomes 7
- uses 7

5 + 7 = 12

4. Integer Wrapper Cache

```
Integer a=100;
Integer b=100;

Integer c=200;
Integer d=200;
```

Answer: B

true
false

Explanation

Java caches Integers from -128 to 127.

100 → cached
200 → new objects

5. Array Reference

```
int[] a={1,2,3};
int[] b=a;

b[1]=10;
```

Answer: B

10

Explanation

Both variables point to the same array.

6. Method Overloading

```
t.show(5);
t.show(5.5f);
```

Answer: B

int double

Explanation

5 → int
5.5f → widened to double

7. String Immutability

String s="Hello";
s.concat(" World");

Answer: B

Hello

Explanation

Strings are immutable.

Need

s = s.concat(" World");

8. equals()

new String("Java")

Answer: B

false
true

Explanation

== → compares addresses

`.equals()` → compares contents

9. Constructor Order

new B();

Answer: A

AB

Explanation

Parent constructor executes first.

10. Static Block

```
static{
System.out.print("A");
}
```

Answer: B

AB

Explanation

Static block executes before `main()`.

11. Array Length

```
new int[5]
```

Answer: B

5

12. Character Arithmetic

```
char c='A';
System.out.println(c+1);
```

Answer: C

66

Explanation

'A' = 65

65+1=66

13. continue

1 2 continue 4 5

Answer: B

1245

14. break

Stops at 3.

Answer: B

12

15. try-catch

10/0

Answer: B

Handled

ArithmeticException is caught.

16. Runtime Polymorphism

A obj=new B();
obj.show();

Answer: B

B

Method overriding is resolved at runtime.

17. StringBuilder

append()

Answer: C

Hi Java

StringBuilder is mutable.

18. Null Reference

String s=null;
s.length();

Answer: B

NullPointerException

19. Integer Division

10/4

Answer: A

2

Fractional part is discarded.

20. Floating Division

10/4.0

Answer: B

2.5

One operand is double.

21. ArrayList

[A,B]
remove("A")

Answer: B

[B]

22. HashSet

10
20
10

Duplicate ignored.

Answer: A

2

23. Enhanced for-loop

for(int x:arr)

Answer: A

4

Explanation

x is only a copy.

Original array doesn't change.

24. switch

```
case 2:
case 3:
```

No break.

Answer: B

BC

25. Final Variable

```
final int x=10;
x=20;
```

Answer: D

Compilation Error

Cannot assign to a final variable.

26. Static Variable

```
new Test();
new Test();
```

Each constructor increments x.

Answer: C

2

27. Static Method Hiding

```
A obj=new B();
obj.show();
```

Answer: A

A

Static methods are resolved using the reference type, not the object type.

28. Auto Unboxing

```
Integer x=10;
int y=x;
```

Answer: A

10

Automatic unboxing.

29. Lambda Expression

```
(x,y)->x+y
```

Answer: B

11

30. Debug Question

String s="Java";

if(s=="Java")

Answer: C

Nothing is wrong; it prints:

Equal

Explanation

Because `"Java"` is a string literal, both references point to the same object in the string pool. While `.equals()` is generally recommended for comparing string contents, == happens to return `true` in this specific case because the references are identical. If s were created using `new String("Java")`, then == would return `false`.